



Vulnerability Assessment Report

Target: OWASP Juice Shop (self-hosted local instance)

Assessment type: Passive (non-intrusive) web application vulnerability assessment

Author: Joshua Cardoz

Date: 18 August 2026

Repository: FUTURE_CS_01 (github.com/joshjcar)

Program: Future Interns — Cyber Security Track, Task 1

1. Executive Summary

This report presents the results of a passive vulnerability assessment of a self-hosted instance of OWASP Juice Shop, a deliberately vulnerable web application used here as an authorized test target.

The assessment identified 10 issues, ranging from Medium to Informational severity. No High or Critical issues were identified — and none were expected — because this was a strictly passive assessment: the application was observed as a normal user would browse it, with no active attacks, injection, or exploitation performed. Confirming higher-severity issues (such as injection or authentication bypass) requires active testing, which was intentionally out of scope for this engagement.

The most significant issues, in business terms, are:

- The application's login token is stored in browser storage that any script on the page can read. If an attacker ever gets a script to run on the site, they can silently steal a user's session.
- The application does not tell the browser which content sources to trust (no Content Security Policy on its main pages). This removes a key defence that would otherwise limit the damage of the issue above.
- The application allows any website to make cross-origin requests to it. This weakens the boundary that normally isolates the application from untrusted third-party sites.

Individually these are moderate. Together they form a single, coherent weakness: the defences that would normally contain a scripting attack are absent, and the credential that a scripting attack would target is left exposed. Addressing them together closes that chain.

The remainder of this report explains each issue in plain business language, provides the technical evidence, classifies the risk, and gives clear remediation steps.

2. Scope and Authorization

- Authorized target: A single, self-hosted instance of OWASP Juice Shop, running locally in a Docker container and bound to the loopback interface (127.0.0.1:3000) so it was never exposed to any external network.
- Legality: OWASP Juice Shop is published specifically for security training and testing. Running and assessing a local instance you control is fully authorized. No third-party or production systems were assessed.
- Assessment window: 18 August 2026.

3. Methodology

This was a passive assessment. The application was exercised the way a legitimate user would (browsing products, registering, logging in, searching, using the basket and feedback forms), while traffic was observed and analysed. No active scanning, fuzzing, injection, or exploitation was performed.

Tools used:

- Nmap — network and service reconnaissance of the host and open port.
- OWASP ZAP (passive mode / Safe Mode) — passive analysis of all observed requests and responses; alerts generated only from traffic the user's own browsing produced.
- curl — direct verification of HTTP response headers at the source, used to confirm or refute tool findings rather than relying on the scanner alone.
- Browser DevTools — inspection of browser storage (localStorage), cookies, and response headers.

Verification principle: where a scanner alert could be ambiguous or affected by out-of-scope traffic, the finding was independently re-verified with curl against the target before inclusion. Alerts that were traced to external domains (unintentionally captured when the test browser touched third-party sites) were excluded from this report.

4. Risk Rating Definitions

- High: Directly exploitable; likely to lead to data loss, account compromise, or system control. (None identified — active testing out of scope.)
- Medium: Meaningfully weakens security; enables or amplifies an attack, or exposes sensitive data under realistic conditions.
- Low: Minor weakness or information leak; limited direct impact but useful to an attacker, or a deviation from best practice.
- Informational: Not a vulnerability in this context; noted for completeness, or applies only in a production configuration.

5. Findings Summary

1. Session/authentication token stored in browser localStorage — Medium
2. Content Security Policy applied inconsistently (absent on main pages, weak where present) — Medium
3. Overly permissive Cross-Origin Resource Sharing (Access-Control-Allow-Origin: *) — Medium
4. Absence of anti-CSRF tokens — Medium
5. Cookie SameSite misconfiguration (None / missing / loosely scoped) — Low to Medium
6. Missing anti-clickjacking protection on some responses — Low to Medium
7. Information disclosure via version banner and verbose error/stack-trace pages — Low to Medium
8. Sub-Resource Integrity (SRI) not used for externally loaded resources — Low
9. Transport session identifier exposed in URL (Socket.IO sid) — Low
10. HTTP Strict Transport Security (HSTS) not set — Informational (applies only to a production HTTPS deployment)

6. Detailed Findings

Finding 1 — Authentication token stored in browser localStorage

- Risk: Medium
- What it is: After a user logs in, the application stores the user's login token (a JWT) in a part of the browser called localStorage. Any JavaScript running on the page — including malicious script introduced through another vulnerability — can read this storage.
- Business impact: If an attacker manages to run script on the site (for example, through a cross-site scripting flaw), they can silently copy the user's login token and impersonate them, without needing their password. This is a common path to account takeover.
- Technical evidence: A token key containing a JWT (eyJ0...) was observed in localStorage for http://localhost:3000 after authentication (Browser DevTools, Storage, Local Storage).
- Reference: CWE-359 (Exposure of Private Information); observed via ZAP passive rule 120000.
- Remediation: Store session tokens in cookies marked HttpOnly, Secure, and SameSite, so that client-side scripts cannot read them. Avoid keeping authentication credentials in localStorage or sessionStorage. Combine with a strong Content Security Policy (Finding 2) to reduce the chance of script injection in the first place.

Finding 2 — Content Security Policy applied inconsistently

- Risk: Medium
- What it is: A Content Security Policy (CSP) tells the browser which sources of scripts, styles, and other content are trusted, and is one of the strongest defences against script-injection attacks. On this application, the main pages send no CSP at all, while certain other responses send a weak policy that permits unsafe practices (unsafe-inline, unsafe-eval, and wildcard sources).
- Business impact: Without a consistent, strict CSP, the browser has no instructions to block untrusted or injected scripts. This significantly increases the impact of any cross-site scripting weakness, and directly amplifies Finding 1.

- Technical evidence: curl requests against the root (/) and an API route (/rest/products/search) returned no Content-Security-Policy header. ZAP separately flagged weak CSP directives (Wildcard Directive, script-src unsafe-eval, style-src unsafe-inline) on other in-scope responses.
- Reference: CWE-693 (Protection Mechanism Failure); observed via ZAP passive rule 10055.
- Remediation: Implement a single, strict CSP across all responses. Avoid unsafe-inline and unsafe-eval; specify explicit trusted sources rather than wildcards; define a default-src fallback. Deliver the policy in enforcing mode (not report-only) once tuned.

Finding 3 — Overly permissive Cross-Origin Resource Sharing (CORS)

- Risk: Medium
- What it is: The application responds with Access-Control-Allow-Origin set to a wildcard (*), meaning it tells browsers that any website is allowed to make cross-origin requests to it.
- Business impact: This weakens the boundary that normally isolates the application from untrusted external sites. Depending on how endpoints are used, it can enable third-party sites to read responses they should not have access to.
- Technical evidence: Access-Control-Allow-Origin: * was observed on http://localhost:3000/scripts.js (ZAP passive rule 10098, Risk Medium). A verbose Access-Control-Allow-Methods list (GET, HEAD, PUT, PATCH, POST, DELETE) was also returned on OPTIONS requests.
- Reference: CWE-264; WASC-14.
- Remediation: Replace the wildcard with an explicit allow-list of trusted origins. Never combine a wildcard Access-Control-Allow-Origin with credentialed requests. Return only the methods actually required.

Finding 4 — Absence of anti-CSRF tokens

- Risk: Medium
- What it is: State-changing requests do not appear to be protected by anti-CSRF (Cross-Site Request Forgery) tokens. CSRF tokens ensure a request genuinely came from the application's own pages, not from a malicious site tricking a logged-in user's browser.
- Business impact: Without this protection, a malicious site could cause a logged-in user's browser to perform actions on the application without the user's intent (for example, changing account details).
- Technical evidence: ZAP passive alert Absence of Anti-CSRF Tokens (3 instances) on in-scope forms.
- Reference: CWE-352 (Cross-Site Request Forgery).
- Remediation: Implement anti-CSRF tokens (synchronizer token pattern) on all state-changing requests, and pair with SameSite cookies (Finding 5) as defence in depth.

Finding 5 — Cookie SameSite misconfiguration

- Risk: Low to Medium
- What it is: Cookies were observed set with SameSite None, without a SameSite attribute, or loosely scoped. The SameSite attribute controls whether cookies are sent on cross-site requests and is part of the defence against CSRF.
- Business impact: Weak cookie scoping increases exposure to cross-site request attacks and can broaden where cookies are sent.

- Technical evidence: ZAP passive alerts Cookie with SameSite Attribute None, Cookie without SameSite Attribute, and Loosely Scoped Cookie.
- Reference: CWE-1275 (Sensitive Cookie with Improper SameSite Attribute).
- Remediation: Set SameSite to Lax (or Strict where feasible), plus Secure and HttpOnly on session cookies. Scope cookies as narrowly as possible (path and domain).

Finding 6 — Missing anti-clickjacking protection on some responses

- Risk: Low to Medium
- What it is: Some responses lack anti-clickjacking protection (X-Frame-Options or CSP frame-ancestors). This protection prevents the site from being embedded inside a hidden frame on a malicious page. Notably, protection is present on some routes but not others — that is, applied inconsistently.
- Business impact: Where protection is missing, an attacker could embed the page invisibly and trick users into clicking on actions they did not intend (clickjacking).
- Technical evidence: ZAP passive alert Missing Anti-clickjacking Header; X-Frame-Options SAMEORIGIN present on the root response but absent on others.
- Reference: CWE-1021 (Improper Restriction of Rendered UI Layers).
- Remediation: Apply X-Frame-Options DENY or SAMEORIGIN, or preferably CSP frame-ancestors, consistently across all responses.

Finding 7 — Information disclosure via version banner and verbose errors

- Risk: Low to Medium
- What it is: The application discloses internal technical details: server and framework version information in headers, and, on error conditions, a full error page containing the framework version and internal source-file paths.
- Business impact: This information helps an attacker fingerprint the technology stack and target it with known, version-specific vulnerabilities. It also reveals internal structure that should not be public.
- Technical evidence: ZAP alerts Server Leaks Information via X-Powered-By and Server Leaks Version Information via Server header. A 500 error response disclosed the framework and version (Express) and an internal path (a build/routes source file with line number).
- Reference: CWE-209 (Information Exposure Through an Error Message).
- Remediation: Suppress or genericise version headers (X-Powered-By, Server). Disable detailed stack traces in production and return generic error pages that do not reveal internal paths or framework versions.

Finding 8 — Sub-Resource Integrity (SRI) not used

- Risk: Low
- What it is: Externally loaded scripts and styles are included without Sub-Resource Integrity (SRI) hashes. SRI lets the browser verify that an externally hosted file has not been tampered with.
- Business impact: If an external or third-party resource were ever compromised, the browser would load the altered file without detection.
- Technical evidence: ZAP passive alert Sub Resource Integrity Attribute Missing.
- Remediation: Add integrity (and crossorigin) attributes to externally loaded resources so the browser rejects modified files.

Finding 9 — Transport session identifier exposed in URL

- Risk: Low
- What it is: A session-style identifier (sid) appears in the URL of the application's realtime (Socket.IO) connection.
- Business impact (honest scoping): ZAP flags this as a Session ID in URL issue. In this case the sid is the Socket.IO / Engine.IO transport connection identifier, not the user's authentication token. Identifiers in URLs can leak via browser history, server logs, or referrer headers, but because this is a transport-layer ID rather than an auth credential, the practical impact is low, and it does not represent session hijacking of a user account.
- Technical evidence: The Socket.IO polling URL exposed a sid parameter (ZAP passive rule 3, parameter sid).
- Reference: CWE-598 (Information Exposure Through Query Strings).
- Remediation: As a general practice, avoid placing session-style identifiers in URLs. Low priority given the identifier type here.

Finding 10 — HTTP Strict Transport Security (HSTS) not set

- Risk: Informational (in this context)
- What it is: The Strict-Transport-Security header is not set. HSTS instructs browsers to only connect over HTTPS.
- Business impact and applicability: HSTS only governs HTTPS connections. This assessment ran against a plain-HTTP local instance, so HSTS is not applicable here and is listed for completeness. On a real production HTTPS deployment, its absence would be a genuine Low-severity finding.
- Technical evidence: ZAP passive alert Strict-Transport-Security Header Not Set.
- Remediation (for production): On the HTTPS production deployment, set Strict-Transport-Security with an appropriate max-age.

7. Positive Observations

Controls the application applied correctly (in scope, on at least the primary responses):

- X-Content-Type-Options nosniff was present, reducing MIME-sniffing risk.
- Anti-clickjacking (X-Frame-Options SAMEORIGIN) was present on the main page (though not applied consistently — see Finding 6).
- The assessment instance was bound to loopback only, correctly limiting network exposure (an operator-side control, noted for completeness).

8. Prioritised Remediation Roadmap

1. Move the auth token out of localStorage into HttpOnly, Secure, SameSite cookies (Finding 1) — closes the highest-value target.
2. Deploy a single strict, enforcing Content Security Policy across all routes (Finding 2) — the biggest single defensive improvement.

3. Restrict CORS to an explicit origin allow-list (Finding 3).
4. Add anti-CSRF tokens and tighten cookie SameSite and scoping (Findings 4 and 5).
5. Apply anti-clickjacking consistently, and suppress version banners and verbose errors (Findings 6 and 7).
6. Add SRI, and treat the URL session identifier and HSTS as low or production-only items (Findings 8, 9, and 10).

9. Limitations and Disclaimer

- This was a passive, non-intrusive assessment. No active scanning, injection, brute-forcing, or exploitation was performed. Findings reflect only what could be observed through normal use.
- Because active testing was out of scope, higher-severity classes of vulnerability (for example, injection or authentication bypass) were neither confirmed nor ruled out. The absence of High findings reflects the assessment method, not a guarantee of their absence.
- The assessment covered a self-hosted local instance used as an authorized training target. Findings are illustrative of the application as configured and are not a statement about any production system.
- One scanner alert regarding a potential cross-site scripting hot-spot was excluded because it was traced to out-of-scope third-party traffic; passive analysis cannot confirm exploitability in any case.

Prepared by Joshua Cardoz — Future Interns Cyber Security Internship, Task 1.